Activity 2: Written Answers

Questions 2, 3 and 4: Propositional Logic
Nurena Babayev · October 22, 2026

Notation and terms used

T = true, F = false. ¬ = not, ∧ = and, ∨ = or, ⇒ = implies, ⇔ = if and only if.

A **model** is one possible world: one way of making every symbol true or false. A sentence is **valid** if it is true in every model, **unsatisfiable** if it is true in no model, and **satisfiable** if it is true in at least one model.

Remember that P ⇒ Q is false in only one case: when P is true and Q is false.

Question 2: Satisfiability

> Decide whether each sentence is valid, unsatisfiable, or neither, and verify the decision using truth tables or the equivalence rules.

How we solve it: we build a truth table that tries every combination of T and F. If the last column is all T the sentence is valid, if it is all F it is unsatisfiable, and if it is a mix it is neither. Then we double-check with the equivalence rules.

2(a) Smoke ⇒ Fire

Answer: Neither. It is satisfiable but not valid.

Truth table. There are two symbols, so there are 2 × 2 = 4 models.

Smoke	Fire	Smoke ⇒ Fire
F	F	**T**
F	T	**T**
T	F	**F**
T	T	**T**

The sentence is true in 3 of the 4 models, so it is satisfiable (not unsatisfiable). It is false in one model, Smoke = T and Fire = F (the shaded row), so it is not valid. Because it is sometimes true and sometimes false, the answer is **neither**.

Check with equivalence rules. By implication elimination, Smoke ⇒ Fire ≡ ¬Smoke ∨ Fire. This "or" is false only when ¬Smoke is false and Fire is false, that is, when Smoke = T and Fire = F. This matches the table.

2(b) ((Smoke ∧ Heat) ⇒ Fire) ⇔ ((Smoke ⇒ Fire) ∨ (Heat ⇒ Fire))

Answer: Valid.

To keep the table readable, call the left side **L =** (Smoke ∧ Heat) ⇒ Fire and the right side **R =** (Smoke ⇒ Fire) ∨ (Heat ⇒ Fire). The sentence is L ⇔ R. There are three symbols, so there are 2 × 2 × 2 = 8 models.

Smoke	Heat	Fire	Smoke ∧ Heat	L	Smoke ⇒ Fire	Heat ⇒ Fire	R	L ⇔ R
F	F	F	F	T	T	T	T	**T**
F	F	T	F	T	T	T	T	**T**
F	T	F	F	T	T	F	T	**T**
F	T	T	F	T	T	T	T	**T**
T	F	F	F	T	F	T	T	**T**
T	F	T	F	T	T	T	T	**T**
T	T	F	T	F	F	F	F	**T**
T	T	T	T	T	T	T	T	**T**

L and R have the same value in every row. They are both false in only one row (Smoke = T, Heat = T, Fire = F, shaded) and both true everywhere else. So L ⇔ R is true in all 8 models, which means the sentence is **valid**.

Check with equivalence rules. Rewrite both sides using only ¬ and ∨:

Left side

(Smoke ∧ Heat) ⇒ Fire

≡ ¬(Smoke ∧ Heat) ∨ Fire — implication elimination

≡ ¬Smoke ∨ ¬Heat ∨ Fire — De Morgan

Right side

(Smoke ⇒ Fire) ∨ (Heat ⇒ Fire)

≡ (¬Smoke ∨ Fire) ∨ (¬Heat ∨ Fire) — implication elimination (twice)

≡ ¬Smoke ∨ ¬Heat ∨ Fire ∨ Fire — associativity, commutativity

≡ ¬Smoke ∨ ¬Heat ∨ Fire — a repeated literal is kept once

Both sides become the same sentence, so L ≡ R. When two sentences are equivalent, L ⇔ R is true in every model. This confirms that the sentence is **valid**.

Summary for Question 2: (a) neither, (b) valid.

Question 3: Models

> The vocabulary has only four propositions, A, B, C and D. How many models are there for each sentence?

Each of the four symbols can be T or F, so there are $2 \times 2 \times 2 \times 2 = 2^4$ = **16 models** in total. For each sentence we count how many of these 16 make it true. Symbols that do not appear in a sentence still count, because they still have to be given a value in every model.

How we solve it: sometimes it is easier to count the models where the sentence is false and subtract them from 16.

3(a) B ∨ C

Answer: 12 models.

B ∨ C is false only when both B and C are false. In that case A and D can still be anything (4 combinations), so there are 4 models where the sentence is false:

A	B	C	D	B ∨ C
F	F	F	F	**F**
F	F	F	T	**F**
T	F	F	F	**F**
T	F	F	T	**F**

All the other models make B ∨ C true: 16 − 4 = **12 models**.

Check by counting directly. B ∨ C is true for 3 of the 4 combinations of B and C (B = F, C = T; B = T, C = F; B = T, C = T). Each of these can be combined with any of the 4 combinations of A and D, so the count is 3 × 4 = 12.

3(b) (A ⇒ B) ∧ A ∧ ¬B ∧ C ∧ D

Answer: 0 models. The sentence is unsatisfiable.

A sentence joined with ∧ ("and") is true only if every part is true. So we need:

A is true	so A = T
¬B is true	so B = F
C is true	so C = T
D is true	so D = T

That leaves only one possible model: A = T, B = F, C = T, D = T. But in this model the first part, A ⇒ B, becomes T ⇒ F, which is **false**.

Part of the sentence	Value when A = T, B = F, C = T, D = T
A ⇒ B	T ⇒ F = **F**
A	**T**
¬B	**T**
C	**T**
D	**T**
whole sentence (all parts joined by ∧)	**F**

Every other model already fails one of the parts A, ¬B, C or D. So no model makes the whole sentence true, and the number of models is **0**.

This also makes sense logically: from A and A ⇒ B we get B (Modus Ponens), but the sentence also says ¬B. B and ¬B cannot both be true, so the sentence contradicts itself.

Summary for Question 3: (a) 12 models, (b) 0 models.

Question 4: Resolution

> Consider the sentence [(Food ⇒ Party) ∨ (Drinks ⇒ Party)] ⇒ [(Food ∧ Drinks) ⇒ Party]

Call the left-hand side **LHS =** (Food ⇒ Party) ∨ (Drinks ⇒ Party) and the right-hand side **RHS =** (Food ∧ Drinks) ⇒ Party. The whole sentence is LHS ⇒ RHS.

How we solve it: (a) check all 8 models in a truth table; (b) rewrite each side step by step into CNF and compare them; (c) assume the sentence is false and show that this leads to a contradiction.

4(a) Enumeration: is it valid, satisfiable (but not valid), or unsatisfiable?

Answer: Valid.

There are three symbols, so there are 8 models. We list all of them:

Food	Drinks	Party	Food ⇒ Party	Drinks ⇒ Party	LHS	Food ∧ Drinks	RHS	LHS ⇒ RHS
F	F	F	T	T	T	F	T	**T**
F	F	T	T	T	T	F	T	**T**
F	T	F	T	F	T	F	T	**T**
F	T	T	T	T	T	F	T	**T**
T	F	F	F	T	T	F	T	**T**
T	F	T	T	T	T	F	T	**T**
T	T	F	F	F	F	T	F	**T**
T	T	T	T	T	T	T	T	**T**

The sentence is true in all 8 models, so it is **valid**. Notice that LHS and RHS are false in exactly the same row (Food = T, Drinks = T, Party = F, shaded) and true in all the others. An implication can only be false when its left side is true and its right side is false, and that never happens here.

4(b) CNF: convert the left and right sides to CNF

CNF (conjunctive normal form) is an "and" of clauses, where each clause is an "or" of literals. The conversion steps are: (1) remove ⇔, (2) remove ⇒ using α ⇒ β ≡ ¬α ∨ β, (3) move ¬ inwards using De Morgan, (4) distribute ∨ over ∧.

Left-hand side

(Food ⇒ Party) ∨ (Drinks ⇒ Party)

≡ (¬Food ∨ Party) ∨ (¬Drinks ∨ Party) — step 2: implication elimination

≡ ¬Food ∨ ¬Drinks ∨ Party ∨ Party — associativity, commutativity

≡ ¬Food ∨ ¬Drinks ∨ Party — a repeated literal is kept once

Steps 1, 3 and 4 are not needed: there is no ⇔, no ¬ in front of brackets, and no ∧.

Right-hand side

(Food ∧ Drinks) ⇒ Party

≡ ¬(Food ∧ Drinks) ∨ Party — step 2: implication elimination

≡ ¬Food ∨ ¬Drinks ∨ Party — step 3: De Morgan

How this confirms (a). Both sides turn into exactly the same clause, ¬Food ∨ ¬Drinks ∨ Party. Call this clause C. The whole sentence is then C ⇒ C, which is equivalent to ¬C ∨ C. In any model either C is true or ¬C is true, so ¬C ∨ C is always true. This means the sentence is valid, which agrees with the enumeration in (a).

In plain words: "food leads to a party, or drinks lead to a party" says the same thing as "food and drinks together lead to a party". Both are false only when there is food and there are drinks but no party. A statement that implies itself can never be false.

4(c) Resolution: prove the answer to (a)

Resolution proves something by contradiction. To show a sentence is valid, we show that its negation is unsatisfiable: we put the negation in CNF and keep resolving clauses until we get the **empty clause** (□), which stands for a contradiction.

Step 1: negate the sentence.

¬(LHS ⇒ RHS)

≡ ¬(¬LHS ∨ RHS) — implication elimination

≡ LHS ∧ ¬RHS — De Morgan, double negation

Step 2: write it in CNF. From (b), LHS is the clause ¬Food ∨ ¬Drinks ∨ Party. For ¬RHS: ¬(¬Food ∨ ¬Drinks ∨ Party) ≡ Food ∧ Drinks ∧ ¬Party (De Morgan, double negation). This gives four clauses:

#	Clause	Comes from
1	¬Food ∨ ¬Drinks ∨ Party	LHS
2	Food	¬RHS
3	Drinks	¬RHS
4	¬Party	¬RHS

Step 3: resolve. Each step takes two clauses that contain a symbol and its negation, cancels that pair, and joins what is left.

#	Resolve	Cancel	New clause
5	1 and 2	¬Food / Food	¬Drinks ∨ Party
6	5 and 3	¬Drinks / Drinks	Party
7	6 and 4	Party / ¬Party	□ (empty clause)

Conclusion. We reached the empty clause, so ¬(LHS ⇒ RHS) is unsatisfiable: it can never be true. Therefore the original sentence can never be false, which proves it is **valid**. This is the same

answer as in (a).

Summary for Question 4: (a) valid; (b) both sides become ¬Food ∨ ¬Drinks ∨ Party, so the sentence has the form C ⇒ C and is always true; (c) the negation resolves to the empty clause, so the sentence is valid.

Summary of answers

Question	Answer	Method used
2(a) Smoke ⇒ Fire	Neither (satisfiable, not valid)	truth table, implication elimination
2(b) ((Smoke ∧ Heat) ⇒ Fire) ⇔ ((Smoke ⇒ Fire) ∨ (Heat ⇒ Fire))	Valid	truth table, equivalence rules
3(a) B ∨ C	12 models	counting models
3(b) (A ⇒ B) ∧ A ∧ ¬B ∧ C ∧ D	0 models (unsatisfiable)	counting models
4(a) enumeration	Valid	truth table with all 8 models
4(b) CNF of both sides	both sides = ¬Food ∨ ¬Drinks ∨ Party	CNF conversion steps
4(c) resolution	Valid (empty clause reached)	resolution on the negation